

CS4231
Parallel and Distributed Algorithms

Lecture 7
Instructor: YU Haifeng

Review of Last Lecture

- Chapter 12 “Message Ordering”
- FIFO ordering for point-to-point messages
 - Already discussed last lecture
- Causal ordering for point-to-point messages
 - Applications
 - Protocol to ensure causal ordering
- Causal ordering for broadcast messages
 - Protocol
- Total ordering for broadcast messages
 - Application
 - Skeen’s algorithm

Today's Roadmap

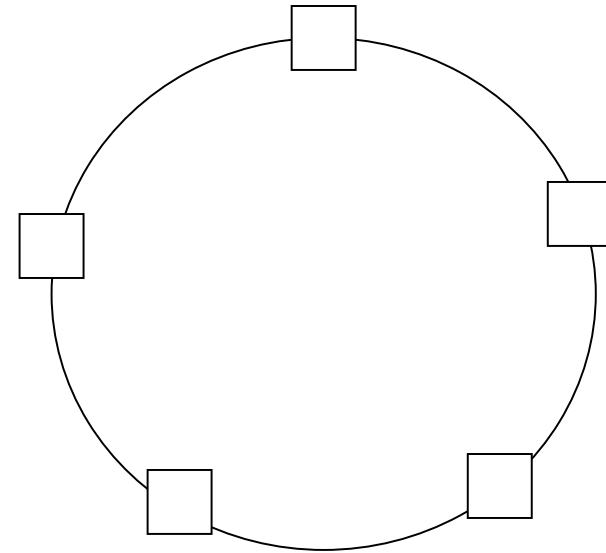
- Chapter 13 “Leader Election”
- Leader election on a ring
- Leader election on a general graph

Motivation

- We often need a coordinator in distributed systems
 - Leader, distinguished node/process
- If we have a leader, mutual exclusion is trivially solved
 - The leader determined who enters CS
 - Inefficient though...
- If we have a leader, totally ordered broadcast trivially solved
 - The leader stamps messages with consecutive integers

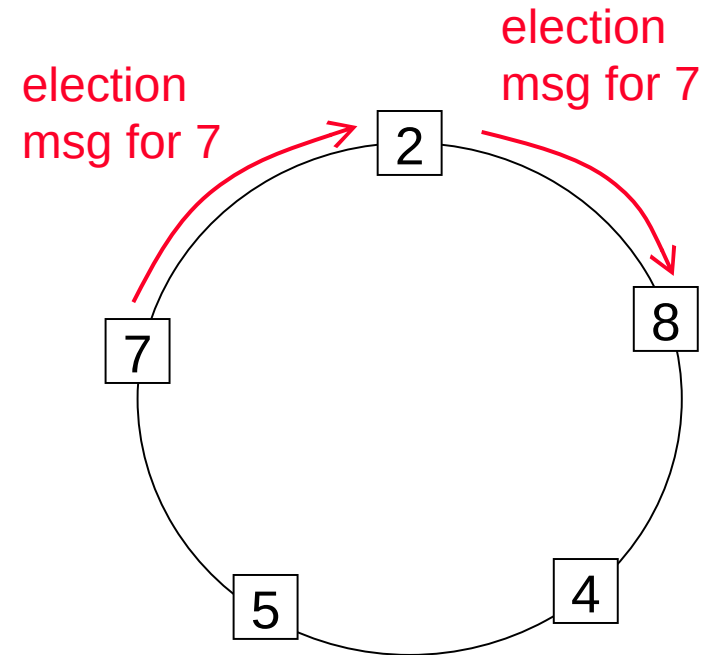
Leader Election on Anonymous Ring

- Anonymous ring:
 - No unique identifiers
- Leader election on anonymous ring is impossible using deterministic algorithms
 - Initial state same
 - Algorithm on each node same
 - Each step is the same
 - Final state the same



Leader Election on Ring: Chang-Roberts Algorithm

- Each node has a unique identifier
- Nodes only send messages clockwise
- **Each node acts on its own**
- Protocol:
 - A node send election message with its own id clockwise
 - Election message is forwarded if id in message larger than own message
 - Otherwise message discarded
- A node becomes leader if it sees its own election message



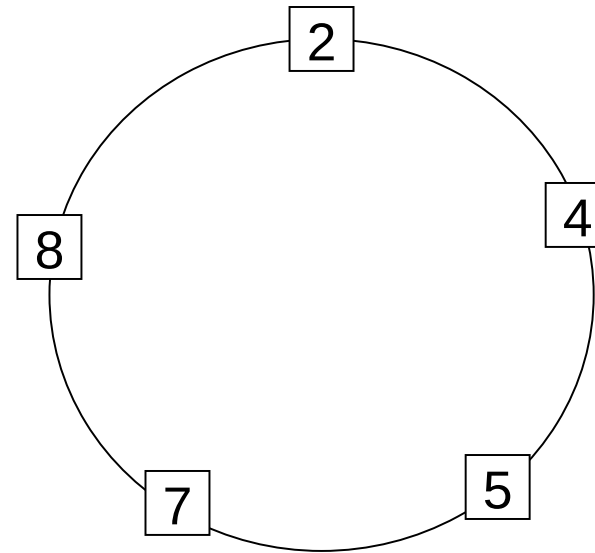
Chang-Roberts Algorithm: Best/Worst Performance

For distributed systems, communication is the bottleneck.
Performance thus is often described as message complexity.

With n nodes on the ring, what is the total number of messages incurred by this protocol, in the best case?

Quick Poll <https://pollev.com/haifengyu229>

- Best case:



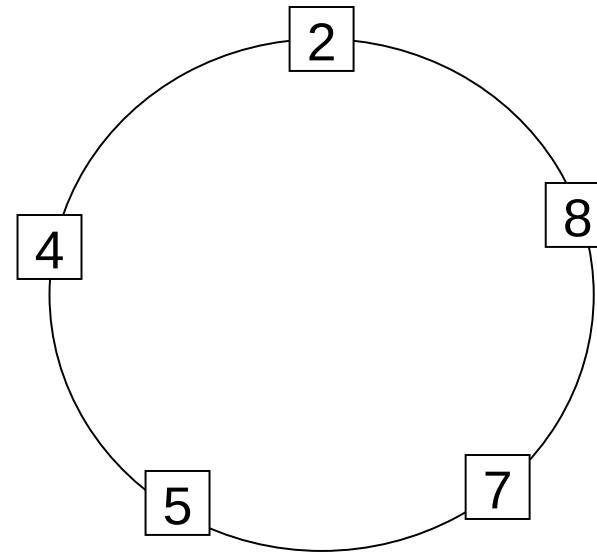
Chang-Roberts Algorithm: Best/Worst Performance

For distributed systems, communication is the bottleneck.
Performance thus is often described as message complexity.

With n nodes on the ring, what is the total number of messages incurred by this protocol, in the worst case?

Quick Poll <https://pollev.com/haifengyu229>

- Worst case:



Chang-Roberts Algorithm: Avg Performance

- The textbook says $O(n \cdot \log(n))$ – no proof is given on textbook
- Average: Taken over all possible orderings of the nodes on the ring, each ordering has the same probability
 - W.l.o.g., assume the n nodes have ids from 1 through n
 - How many total orderings (exclude isomorphic ones)?

$$n! / n = (n - 1)!$$

- Define random variable X_k : Number of messages caused by the election message from node k
- We want to calculate $E[\sum x_k]$ for k summed from 1 to n , note:

$$E[\sum x_k] = \sum E[x_k] \quad \text{by linearity of expectation}$$

- Now consider a given k :

$$E[x_k] = \sum_{i=1}^k (i \cdot \Pr[x_k = i])$$

While possible to calculate brute-force, will be messy!

We will use the following tricks:

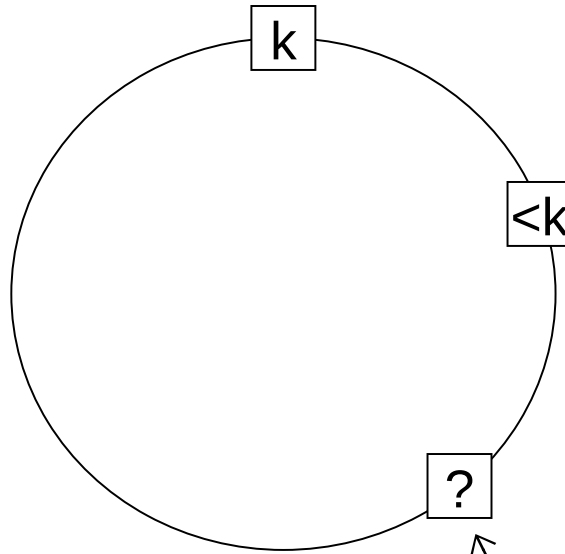
- Claim 1: $\Pr[x_k = 1] = \frac{(n - k)}{(n - 1)}$

Reason: X_k is the number of msgs caused by node k .

Total $n - 1$ nodes other than node k ,

out of which $n - k$ nodes have ids larger than k .

- Claim 2: $\Pr[x_k = 2 | x_k > 1] = \frac{(n - k)}{(n - 2)}$



For the event to happen, this id needs to be larger than k

Total $n - 2$ IDs left, out of which $n - k$ IDs are larger than k

What we have got so far:

$$\Pr[x_k = 1] = \frac{(n - k)}{(n - 1)}$$

$$\Pr[x_k = 2 | x_k > 1] = \frac{(n - k)}{(n - 2)}$$

$$\Pr[x_k = 3 | x_k > 2] = \frac{(n - k)}{(n - 3)}$$

...

$$\Pr[x_k = k | x_k > (k - 1)] = \frac{(n - k)}{(n - k)} = 1$$

We want to simplify further. Define: $p = \frac{(n - k)}{(n - 1)}$

$$\Pr[x_k = 1] = \frac{(n - k)}{(n - 1)} \geq p$$

$$\Pr[x_k = 2 | x_k > 1] = \frac{(n - k)}{(n - 2)} \geq p$$

$$\Pr[x_k = 3 | x_k > 2] = \frac{(n - k)}{(n - 3)} \geq p$$

...

$$\Pr[x_k = k | x_k > (k - 1)] = \frac{(n - k)}{(n - k)} = 1 \geq p$$

Reminder: Our goal is
to calculate $E[x_k]$

Chang-Roberts Algorithm: Avg Performance

- Consider the following experiment:
 - We buy lottery tickets, each ticket has a winning probability p
 - Let y be the random variable denoting the number of tickets we buy until we win for the very first time

$$\Pr[y = (i + 1) | y > i] = p$$

- What is the expectation of y ?

$$E[y] = 1 / p$$

- Now compare X_k and y :
 - The chance winning for X_k is always no smaller than the winning chance of y , thus:

$$E[x_k] \leq E[y] = \frac{1}{p} = \frac{(n - 1)}{(n - k)}$$

Chang-Roberts Algorithm: Avg Performance

$$\sum_{k=1}^n E[X_k] = n + \sum_{k=1}^{n-1} E[X_k]$$

$$< n + \sum_{k=1}^{n-1} \frac{n-1}{n-k} = n + (n-1) \sum_{k=1}^{n-1} \frac{1}{k}$$

$$= n + (n-1)O(\log n) = O(n \log n)$$

Leader Election on General Graph (n known)

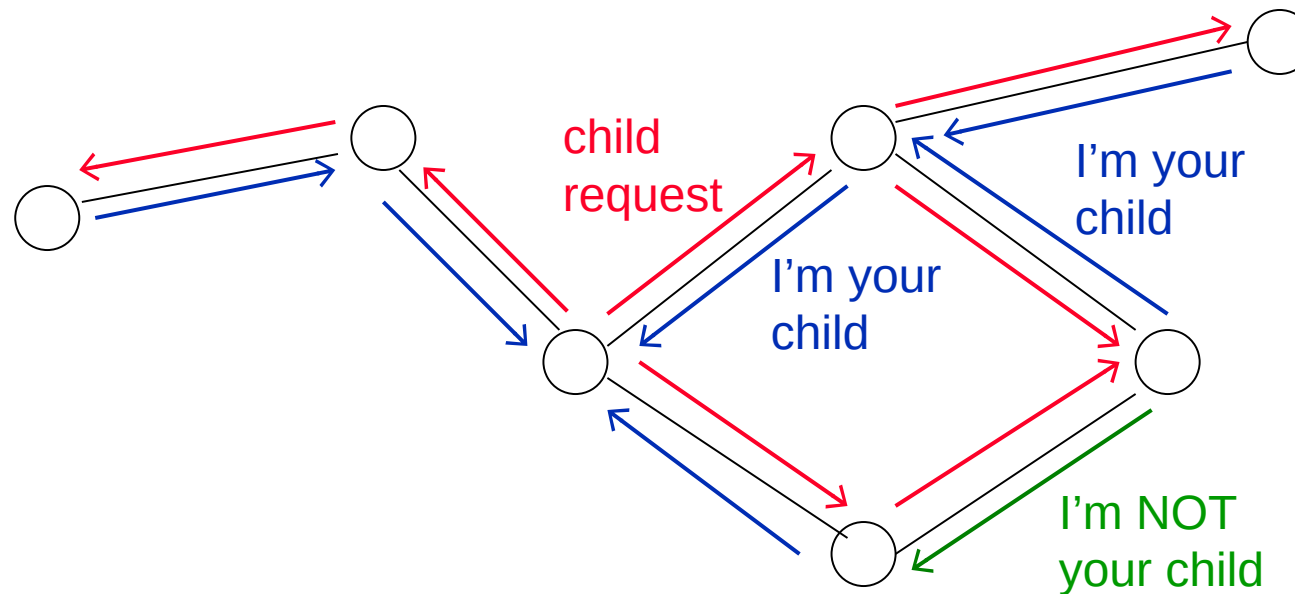
- Complete graph
 - Each node send its id to all other nodes
 - Wait until you receive n ids
 - Biggest id wins
- Any connected graph
 - **Flood** your id to all other nodes (how?)
 - Wait until you receive n ids
 - Biggest id wins

Leader Election on General Graph (n unknown)

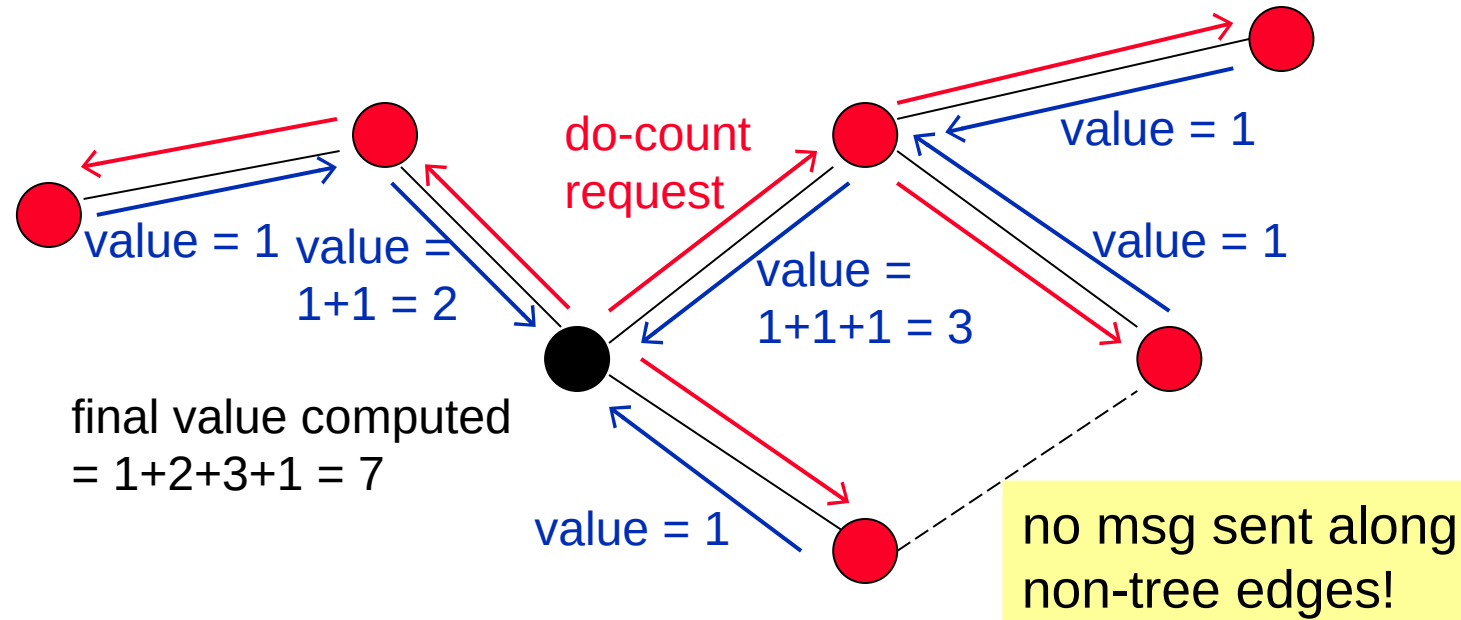
- Complete graph
 - n must be known!
- Any connected graph
 - An auxiliary protocol to calculate the number of nodes
- The protocol is initiated by any node who wants to know n
 - The protocol actually establishes a spanning tree starting from the initiator

Spanning Tree Construction

- Remember: No centralized coordinator
 - Goal of the protocol: Each node knows its parent and children (i.e., a distributed tree)



Counting Nodes Using a Spanning Tree



Spanning Tree Useful for Other Purposes

- Broadcast
 - The root does not need to send out n message
- Aggregation
 - Average temperature sensed by distributed sensors
 - Summation and number of nodes
- Calculation max, min

Summary

- Chapter 13 “Leader Election”
- Leader election on a ring
 - Chang-Roberts Algorithm
 - Interesting avg performance analysis
- Leader election on a general graph
 - Spanning tree construction

Homework Assignment

- Consider an anonymous ring where ring size is unknown to the nodes. Can you construct a randomized algorithm to elect a leader on this ring?
 - Your algorithms must be correct in all cases (i.e., never elects more than one leader)
 - Clearly explain how you distinguish messages in one round from the other (i.e., they should not interfere with each other)
 - Clearly explain why you think the leader is unique
 - Clearly explain why you think the protocol finishes with probability 1.
- Homework due a week from today
- Read Chapter 15